Shared-Memory SPSC IPC: A Controlled Study of Wakeup Mechanisms and SQPOLL-Assisted Signaling

Rahul Raman
Department of Computer Science and Engineering
International Institute of Information Technology Bangalore
Bangalore, India

Yuvraj Deshmukh
Department of Computer Science and Engineering
International Institute of Information Technology Bangalore
Bangalore, India

B. Thangaraju
Department of Computer Science and Engineering
International Institute of Information Technology Bangalore
Bangalore, India

***Abstract*—Inter-Process Communication (IPC) performance is a foundational bottleneck in concurrent, multi-process software systems. Traditional kernel-mediated mechanisms—POSIX Pipes, UNIX Domain Sockets, and POSIX Message Queues— require system calls and kernel-mediated data movement and may incur scheduler transitions; in our depth-1 suite, their 64 B median single-trip latencies are 2.71–2.86 μs. Modern high-performance systems reduce this cost by employing lock-free Single-Producer Single-Consumer (SPSC) shared-memory ring buffers that transfer payload data entirely in userspace via memcpy into POSIX shared memory regions. A critical and largely unanswered question, however, is: when the shared ring empties and the consumer must sleep, which wakeup primitive should be used—and what does Linux's io_uring framework specifically contribute over simpler alternatives such as futex or eventfd?**

This paper presents a rigorous empirical measurement study that decouples payload transport from wakeup-mechanism signaling and applies a two-suite evaluation framework: (1) a wakeup-mechanism ablation comparing six coordination strategies (busy_poll, spin_backoff, adaptive, futex, eventfd, io_uring) on an identical cache-aligned SPSC ring under saturated, bursty, and offered-load traffic conditions across a 64 B–1 MiB message-size sweep; and (2) a corrected depth-1 ping-pong latency suite in which both the send and receive timestamps are recorded on the single initiator core using CLOCK_MONOTONIC_RAW, eliminating cross-core TSC drift and in-flight pipelining artefacts, reporting full percentile distributions (P50, P90, P99, P99.9) using size-scaled samples of 2 000–100 000 round-trips per supported configuration.

Our results separate the roles of the data and wakeup paths. At 64 B, adaptive and busy-poll waiting achieve median single-trip latencies of 0.211 and 0.185 μs, respectively; the pipe and socket medians are 12.8–15.0$\times$ higher. In the controlled saturated ablation, the six primary variants span 13.37–15.75 GiB/s at 64 KiB, with interrupt-mode io_uring at 13.37 GiB/s. In depth-1 ping-pong tests (Suite 2), io_uring incurs a modest per-event ring submission overhead (+1.61 μs over futex at 64 B). However, under bursty wakeup conditions (Suite 1), where wakeups occur when the ring empties, io_uring performs on par with futex and eventfd (1.092–1.095 ms median wakeup latency). Thus, the shared-memory data path dominates saturated throughput, whereas the wait strategy dominates empty-ring latency and CPU cost. A frequency-pinned replication (`performance` governor and `no_turbo=1`) reproduces the close ordering of the kernel-sleeping variants. We additionally evaluate SQPOLL as a separate configuration: its 64 B bursty wakeup median (1058.5 μs) is close to interrupt-mode io_uring (1095.0 μs), rather than a decisive improvement.

***Index Terms*—io_uring, IPC, shared memory, SPSC ring buffer, lock-free, wakeup mechanism, latency, throughput, Linux kernel, systems performance**

I. Introduction

Modern software infrastructure—spanning high-frequency trading (HFT) matching engines, real-time database kernels, media processing pipelines, and containerised microservice mesh proxies—relies heavily on efficient Inter-Process Communication. Even a modest per-message overhead of a few microseconds accumulates to tens of milliseconds when millions of messages flow per second. At 100 kmsg/s, a 3 μs per-message overhead consumes approximately 30 % of one CPU core; at 10 Mmsg/s it would require about 30 core-seconds of work per second.

Linux provides a rich set of mature IPC primitives. POSIX FIFOs and UNIX domain sockets route all data through the kernel, imposing context switches and double memory copies on every message. POSIX message queues add priority ordering but retain kernel-mediated delivery semantics. Shared memory avoids the data-copy penalty by mapping identical physical pages into both producer and consumer address spaces; the kernel is contacted only for initial setup.

The cost of these kernel interactions is non-trivial and grows with message rate. A `write`/`read` pair introduces fixed entry, copy, and scheduling costs before payload-dependent work is considered. At high message rates, these per-message costs can consume several cores and become the dominant budget for latency-sensitive applications.

Lock-free SPSC ring buffers—popularised by the LMAX Disruptor [5] and Intel DPDK [6]—take shared memory a step further by coordinating access using only atomic load/store instructions on cache-line-aligned indices, avoiding any kernel involvement on the data path. Message delivery in the uncontended case reduces to a `memcpy` plus two atomic index updates; the measured adaptive P50 is 211 ns at 64 B, more than an order of magnitude below the blocking baselines in this experiment.

A remaining challenge is the *empty-ring wakeup problem*: when the ring is empty, busy-spinning wastes a full CPU core; sleeping the consumer requires a kernel-level wakeup mechanism.

Linux's `io_uring` interface (introduced in kernel 5.1 [1]) has attracted substantial interest for its promise of zero-syscall I/O. A growing body of practitioner literature presents "`io_uring`-based IPC rings" as offering superior latency and throughput over both traditional IPC and simpler shared-memory approaches. However, these claims conflate two independent design decisions: (1) the payload data path choice (shared memory vs. kernel-copy), and (2) the wakeup mechanism choice (`io_uring` vs. `futex` vs. spin). No prior measurement study isolates these dimensions on identical hardware with a controlled ablation.

Contributions

1) **Controlled wakeup ablation.** Six pluggable wakeup strategies are implemented on a bit-identical SPSC ring buffer; their impact on latency, throughput, and CPU time per message are measured across a 64 B–1 MiB payload sweep.
2) **Corrected depth-1 ping-pong protocol.** Queue depth is enforced at 1 and both round-trip endpoints are timestamped on the same core with `CLOCK_MONOTONIC_RAW`, eliminating TSC skew and pipelining artefacts.
3) **Tail-latency characterisation.** P50, P90, P99, P99.9, and 95 % confidence intervals are reported for every supported payload size, including 64 KiB (10 000 rounds) and 1 MiB (2 000 rounds). The 64 B and 4 KiB configurations use 100 000 and 50 000 rounds, respectively.
4) **Explicit measurement scope.** We report interrupt-mode io_uring separately from the SQPOLL configurations and label the Suite 2 implementation as SQPOLL-assisted signaling because its blocking read/wait rings remain interrupt-mode. Fixed-frequency settings are recorded for the low-load wakeup measurements.
5) **Reproducible artifact.** All source code, raw CSV data, and figure-generation scripts are published at https://github.com/Rahul09123/io_uring-IPC.

II. Background & Related Work

A. Traditional Kernel-Mediated IPC

POSIX Pipes (FIFOs) (Stevens and Rago [2]; Linux man-pages [10]) provide uni-directional byte streams backed by a kernel circular buffer (default 64 KiB, tunable to 1 MiB via `fcntl(F_SETPIPE_SZ)`). Every `write` copies data from user to kernel space; every `read` copies it back. The producer and consumer alternate between running and sleeping states as the pipe buffer fills and drains, incurring scheduler-wakeup latency on each transition. At very small messages (64 B), the pipe's fixed per-syscall entry cost dominates; at large messages (≥256 KiB), finite pipe capacity creates producer backpressure.

UNIX Domain Sockets [10] bypass the network stack but retain the socket buffer abstraction (`sk_buff`). Each `sendmsg`/`recvmsg` pair requires a system call and a socket-layer serialisation lock (`mutex_lock` visible in flamegraph traces). Socket send/receive buffers are tuned to 2 MiB via `SO_SNDBUF`/`SO_RCVBUF` in our implementation. Despite the per-call overhead, UNIX sockets achieve competitive throughput at large payload sizes in the recorded runs. The experiment does not isolate which socket-buffer or flow-control behavior produces that result.

TABLE I
Kernel-IPC System-Call and Copy Costs per Message

Mechanism	Syscalls /msg	Copies /msg	Kernel buffer
POSIX Pipe	2	2	Ring buf (64 KiB–1 MiB)
UNIX Socket	2	2	`sk_buff` (up to 2 MiB)
POSIX MQ	2	1–2	VFS inode (10 msgs)
SHM Ring	0	1	None (userspace)

POSIX Message Queues (`mqueue`) [10] deliver structured, priority-sorted messages via a virtual filesystem inode under `/dev/mqueue`. A key optimisation—`pipelined_send`—delivers directly into a blocked receiver's address space when one is already waiting, bypassing intermediate queue staging and making `mqueue` competitive with shared-memory rings at small message sizes. However, the default maximum message size is 8 KiB and the maximum queue depth is 10 messages, both controlled by kernel tunables (`fs.mqueue.msgsize_max`, `fs.mqueue.msg_max`), imposing hard architectural limits at scale.

B. Lock-Free SPSC Shared-Memory Rings

Shared-memory IPC maps identical physical pages into producer and consumer address spaces via `shm_open` and `mmap`. SPSC ring buffers (Lamport [4], Herlihy and Shavit [9], LMAX Disruptor [5], DPDK `rte_ring` [6]) coordinate using a pair of atomic integer indices. The critical correctness rule is: the producer advances `head` with `release` ordering after writing payload; the consumer reads `head` with `acquire` ordering before consuming.

The C++17 memory model formally defines these orderings. A *release store* ensures all prior writes are visible to any thread that subsequently performs an *acquire load* of the same variable. For an SPSC ring, this means the consumer observing `head > tail` is guaranteed to see the complete payload written by the producer before `head` was advanced. Using `memory_order_relaxed` on `head` (a common optimisation) may appear to work on x86 due to Total Store Order (TSO), but does not provide the release/acquire ordering required by this algorithm on weakly ordered architectures (ARM, RISC-V, Power). The resulting stale-payload or lost-wakeup logic error is possible where Store-Load reordering is permitted. We use `memory_order_seq_cst` on the publish store to maintain portability and prevent the lost-wakeup race.

False sharing—where producer and consumer cache lines collide—is prevented by placing `head`, `tail`, and control flags on separate 64-byte-aligned cache lines using `alignas(64)`. Without this separation, a producer store to `head` invalidates the cache line containing `tail` on the consumer core, forcing a cache-coherency RFO (Request For Ownership) on every slot exchange and doubling effective memory latency.

C. The io_uring Interface

Introduced by Jens Axboe [1] and merged in Linux 5.1, `io_uring` exposes two ring buffers shared between userspace and the kernel: the Submission Queue (SQ) and the Completion Queue (CQ). In the default interrupt-driven mode (`flags=0`), a single `io_uring_enter` system call submits all pending SQEs and optionally waits for completions. In SQPOLL mode (`IORING_SETUP_SQPOLL`), a dedicated kernel poller thread can eliminate submission syscalls; completion waiting and host policy still determine whether other kernel entries occur.

In our implementation, `io_uring` is used *exclusively for the wakeup signaling path*. In Suite 1, the consumer first applies a 10 ms `poll()` readiness gate to the named FIFO and then submits an `IORING_OP_READ`; the tested wait path is therefore `poll`/`io_uring`, not a pure syscall-free wait. When the producer detects `consumer_sleeping` = 1, it submits an `IORING_OP_WRITE` to the same FIFO. The payload data path is pure `memcpy` into shared memory—`io_uring` never touches message bytes.

D. Prior Work on io_uring Performance

Didona et al. [7] compared `libaio`, SPDK, and `io_uring` for storage I/O, demonstrating that polling makes `io_uring` competitive with SPDK for sequential workloads. Ren and Trivedi [8] demonstrated that `io_uring` can match kernel-bypass performance for certain NVMe access patterns. Neither study addresses the question of wakeup-primitive selection for shared-memory IPC, leaving this dimension unexplored. Our work fills this gap with a controlled ablation study on a single fixed ring design.

III. System Design

Our architecture cleanly separates the data path from the control path.

A. Lock-Free SPSC Ring Buffer Layout

The shared ring buffer is a single POSIX shared memory object opened via `shm_open` and mapped with `mmap` into both the producer and consumer address spaces.

Listing 1. Cache-aligned SPSC RingBuffer (common.h)

```
struct alignas(64) RingBuffer {
  alignas(64) atomic<uint64_t> head;   // Producer write
     index
  alignas(64) atomic<uint64_t> tail;   // Consumer read
     index
  alignas(64) atomic<uint32_t> consumer_sleeping;

  struct alignas(64) Slot {
    uint64_t send_ns;           // Producer timestamp
     (Suite 1 clock)
    uint64_t wakeup_publish_ns; // Wakeup-signal timestamp
    uint32_t size;              // Payload bytes written
    char     data[MAX_PAYLOAD]; // Up to 1 MiB of payload
  };
  Slot slots[NUM_SLOTS];        // Fixed 64-slot ring
     (~64 MiB total)
};
```

Three design properties are critical.

(1) Cache-line isolation. `head`, `tail`, and the sleeping control flag each occupy their own 64-byte cache line. Without this separation, a producer write to `head` invalidates the cache line containing `tail` on the consumer core, forcing an RFO (Request For Ownership) round-trip on every slot transition. Our hardware counter data (Table IX) are consistent with lower data-path cache pressure: the SHM Ring's 4.97 % LLC miss rate compares favourably to 30–34 % for the measured pipe and socket baselines.

(2) Sequential consistency at publish boundaries. `head` is stored with `memory_order_seq_cst` to prevent Store-Load reordering that could cause the producer to read a stale cleared sleeping flag and skip sending a wakeup, silently deadlocking the consumer. Compilers enforce the required global ordering with architecture-specific locked or fenced sequences; the exact instruction sequence is compiler- and target-dependent.

(3) Lost-wakeup prevention. A classic race exists in any producer-consumer pair: the consumer checks `head == tail` (empty), decides to sleep, but before setting the sleeping flag, the producer writes a slot and finds the flag clear (decides not to signal), then the consumer sets it and waits—forever. We eliminate this race by having the consumer set the sleeping flag *first*, then re-check `head != tail` under a full memory fence. If a slot arrived in the window, the re-check catches it and the consumer proceeds without sleeping.

Data-flow walkthrough. On every message transfer: (a) the producer calls `memcpy` into `slots[head % NUM_SLOTS].data`, writes the timestamp and size, then executes a seq_cst store to `head`; (b) the consumer spin-reads `head` with acquire ordering; once it observes `head > tail`, it processes the slot and advances `tail` with a release store; (c) if the consumer finds the ring empty after its deadlock-prevention recheck, it invokes the configured wakeup primitive to sleep. Steps (a) and (b) involve zero system calls; step (c) invokes the kernel calls required by the selected wait path.

B. Pluggable Wakeup Layer: Six Ablation Variants

To isolate the contribution of the wakeup mechanism, we implement six interchangeable wakeup modules on a single, bit-identical SPSC ring buffer data structure. Each primitive is invoked via its standard Linux kernel system interface (Linux man-pages [10]): Only the functions `consumer_wait()` and `producer_signal()` differ, guaranteeing that any observed latency difference is attributable solely to the wakeup mechanism.

a) 1. `busy_poll`.: The consumer spins in a tight `while(head.load(relaxed) == tail)` loop with no backoff. Zero system calls are issued while waiting. The mea-

TABLE II
WAKEUP MECHANISM PROPERTIES SUMMARY

Variant	Wait Strategy	Idle Behavior	Kernel Wait Path
`busy`	Tight spin	Continuous spin	None
`spin+back.`	Spin+PAUSE	Continuous spin	None
`adaptive`	Spin-then-futex	Hybrid	Futex
`futex`	FUTEX_WAIT	Sleeping	Futex
`eventfd`	poll+read	Sleeping	poll/eventfd
`io_uring`	poll+submit/wait	Sleeping	poll/io_uring

sured 64 B depth-1 P50 is 0.185 μs, at the cost of continuously occupying the pinned consumer core.

b) 2. `spin_backoff`.: Same tight spin with an `_mm_pause()` intrinsic on every iteration. `_mm_pause` inserts a brief pipeline hint on x86 to make sustained spinning friendlier to the execution pipeline; this study does not measure package power directly.

c) 3. `adaptive`.: The consumer spins for up to 500 iterations; if no data arrives, it falls back to a `futex` sleep. This hybrid targets low-latency workloads where the ring is rarely empty (spin wins) while gracefully handling burst-idle traffic (the blocking phase reduces busy-wait CPU time; power was not measured).

d) 4. `futex`.: The consumer sets the sleeping flag to 1 and calls `SYS_futex(FUTEX_WAIT)`. The kernel checks the address atomically; if it still reads 1, the thread is suspended. The producer stores 0 and calls `FUTEX_WAKE`. One system call per sleep and per wakeup event.

e) 5. `eventfd`.: The consumer `poll`s an eventfd file descriptor and then `read`s to consume the wakeup token. The producer `write`s 1 to the eventfd. The eventfd is compatible with `epoll`/`select`, easing integration into existing event-loop frameworks.

f) 6. `io_uring`.: The consumer first calls `poll()` with a 10 ms timeout on a named FIFO (`/tmp/uring_sig_fifo`); after readiness, it submits an `IORING_OP_READ` SQE and waits through `io_uring_submit_and_wait(ring, 1)`. The producer submits an `IORING_OP_WRITE` SQE to the same FIFO and calls `io_uring_submit`. The `io_uring` context can simultaneously monitor additional FDs, making it advantageous for frameworks managing many concurrent channels. Our primary ablation evaluates standard interrupt mode (`flags=0`); additionally, the implementation provides runtime support for `IORING_SETUP_SQPOLL` (SQPOLL mode via `USE_SQPOLL=1`), which offloads ring polling to an asynchronous kernel thread when the host permits SQPOLL operation.

IV. EXPERIMENTAL METHODOLOGY

A. Two-Suite Benchmark Framework

Suite 1 — Wakeup-Mechanism Ablation. The producer operates under three regime families: *saturated* (back-to-back sends), *bursty* (64-message bursts separated by 1 ms), and offered-load sweeps at 25, 50, 75, and 90 % of saturation. Payload sizes sweep across {64 B, 256 B, 1 KiB, 4 KiB, 16 KiB, 64 KiB, 256 KiB, 1 MiB}. $N = 15$ measured runs per configuration (one warmup discarded). Suite 1 timestamps are collected on the pinned producer and consumer with `std::chrono::high_resolution_clock`.

TABLE III
REFERENCE TEST ENVIRONMENT

Parameter	Value
System	ASUS Vivobook K3605ZF
OS / Kernel	Ubuntu Linux / 6.17.0-35-generic
CPU	Intel Core i5-12500H (Alder Lake, 12th Gen)
Architecture	Hybrid P/E-core
Cores/Threads	12 Physical / 16 Logical
Rated Frequency	400–4500 MHz; Turbo disabled during runs
L1d Cache	448 KiB (12 instances)
L2 Cache	9 MiB (6 instances)
L3 Cache	18 MiB (shared)
RAM	16 GiB DDR4-3200
Storage	512 GB Samsung NVMe SSD
Compiler	g++ -O3, C++17 (ablation) / C++20 (ping-pong)
liburing	System package, Ubuntu 24.04
CPU Governor	`performance`; `intel_pstate.no_turbo=1`

Suite 2 — Corrected Depth-1 Ping-Pong. Queue depth is strictly enforced to 1: the initiator sends one message and blocks waiting for the echo server to reflect it before sending the next. Unlike Suite 1, both latency endpoints are timestamped on one core. Both t_{start} and t_{end} are recorded on **Initiator Core 1** using `clock_gettime(CLOCK_MONOTONIC_RAW)`. Single-trip latency:

$$L_{\text{trip}} = \frac{t_{\text{end}} - t_{\text{start}}}{2} \ [\mu\text{s}]$$

The suite discards 10 000 warmup trips and uses size-scaled measured samples: 100 000 rounds through 1 KiB, 50 000 at 4 KiB, 20 000 at 16 KiB, 10 000 at 64 KiB, 5 000 at 256 KiB, and 2 000 at 1 MiB. P50, P90, P99, P99.9, and a 95 % confidence interval are reported.

B. Hardware and System Configuration

Core affinity. Producer/Initiator pinned to Core 1; Consumer/Echo-Server pinned to Core 2 via `sched_setaffinity`. Both cores share the 18 MiB L3 cache, keeping the ring buffer warm across the inter-process boundary. The environment records the logical CPU IDs but not their P-core/E-core classification; results therefore apply only to this pinned pair on the hybrid Alder Lake host.

CPU tuning and frequency-pinned replication. The final ablation and ping-pong runs were collected on the native Linux host with Core 1 and Core 2 pinned, the `performance` governor on both cores, and `intel_pstate.no_turbo=1`. These settings are recorded in `data/environment_ablation.txt` and `data/environment_pingpong_primary.txt`. Core isolation through `isolcpus`/`nohz_full` was not used; residual scheduler and idle-state effects therefore remain a limitation.

TABLE IV
SUITE 1 DATA VOLUME PER RUN

Configuration	≤1 KiB	4–64 KiB	≥256 KiB
Each wakeup variant	16 MiB	128 MiB	512 MiB

SQPOLL-assisted signal-mode run. The completed Suite 2 SQPOLL configuration was run on the reference native Linux host, with logical CPUs 1 and 2 pinned, the `performance` governor, and `intel_pstate.no_turbo=1`; its record is `data/environment_pingpong.txt`. It establishes that the SQPOLL assisted signaling configuration completes under the documented protocol. The blocking FIFO read/wait rings remain interrupt-mode to avoid a depth-1 SQPOLL deadlock, so this is not a fully syscall-free end-to-end configuration.

C. Statistical Methodology

Following established systems-performance measurement practice [3], we report central tendency, tail percentiles, and uncertainty rather than a single best run. For each (mechanism, payload size) pair we compute:

$$\bar{T} = \frac{1}{N}\sum_{k=1}^{N} T_k \ \ [\text{GiB/s}], \quad \bar{L} = \frac{1}{M}\sum_{i=1}^{M} L_i \ \ [\mu\text{s}]$$

$$\sigma_L = \sqrt{\frac{1}{M}\sum_{i=1}^{M}(L_i - \bar{L})^2}$$

where $N = 15$ is the Suite 1 run count and M is the number of measured round trips in Suite 2. Ping-pong 95 % confidence intervals use $\bar{L} \pm 1.96\sigma_L/\sqrt{M}$, matching the published analysis code.

D. Workload Volumes

V. EMPIRICAL RESULTS

A. Unloaded Ping-Pong Latency (Suite 2)

Table V reports single-trip latency distributions (P50, P90, P99, P99.9, and mean ± 95% CI) at 64 B (100 000 rounds), 4 KiB (50 000 rounds), 64 KiB (10 000 rounds), and 1 MiB (2 000 rounds). Values are drawn from the canonical root `data/pingpong_*_summary.csv` files.

Key observations:

(1) 12.1–13.8× spinning advantage at small messages. At 64 B, `busy_poll` (0.185 µs) and `adaptive` (0.211 µs) are at least 12× lower than any kernel-sleeping variant; busy polling is 13.8× lower than futex. This gap includes the system call, scheduler wakeup, and context-switch path needed to resume a sleeping consumer.

(2) Kernel-sleeping variants remain in the same latency class. `futex` (2.562 µs), `eventfd` (2.602 µs), and `io_uring` (4.173 µs) span 2.6–4.2 µs P50 at 64 B. The higher `io_uring` value is consistent with additional SQ/CQ ring management. These figures come from the canonical 100,000-trip distributions.

TABLE V
SINGLE-TRIP PING-PONG LATENCY DISTRIBUTIONS (µS). SAMPLE COUNTS ARE 100 000 (64 B), 50 000 (4 KiB), 10 000 (64 KiB), AND 2 000 (1 MiB). EACH ROW REPORTS P50, P90, P99, P99.9, AND MEAN ± 95% CI FROM THE CANONICAL ROOT SUMMARY CSVS. SQPOLL DENOTES THE SEPARATELY LABELLED SQPOLL-ASSISTED SIGNALING CONFIGURATION.

Transport / Variant	P50	P90	P99	P99.9	Mean ± CI
Message Size: 64 B — lower is better					
shm (busy_poll)	**0.185**	0.227	0.235	0.502	0.191 ± 0.001
shm (adaptive)	0.211	0.244	0.260	0.612	0.216 ± 0.001
shm (spin_back)	0.375	0.634	0.679	1.993	0.417 ± 0.001
shm (futex)	2.562	2.619	3.457	4.663	2.589 ± 0.002
shm (eventfd)	2.602	3.275	4.121	5.795	2.727 ± 0.002
pipe	2.710	2.773	3.787	5.827	2.747 ± 0.002
posix_mq	2.861	2.920	3.864	4.936	2.901 ± 0.012
unix_socket	2.770	4.694	5.534	7.555	3.271 ± 0.006
shm (io_uring)	4.173	5.275	6.156	9.433	4.422 ± 0.004
shm (io_uring SQPOLL)	6.912	7.051	9.417	1485.060	13.816 ± 0.570
Message Size: 4 KiB — lower is better					
shm (adaptive)	**1.124**	1.160	1.219	2.867	1.130 ± 0.001
shm (busy_poll)	1.156	1.192	1.239	2.757	1.155 ± 0.001
shm (spin_back)	1.335	1.373	1.471	3.000	1.330 ± 0.001
shm (eventfd)	3.398	3.574	4.357	6.071	3.467 ± 0.030
shm (futex)	3.390	4.053	5.023	8.107	3.603 ± 0.064
pipe	3.634	3.824	4.937	7.293	3.719 ± 0.012
posix_mq	4.408	4.580	5.597	7.063	4.469 ± 0.021
shm (io_uring)	5.004	5.156	6.795	8.765	5.153 ± 0.092
unix_socket	6.063	6.428	7.118	10.059	6.058 ± 0.011
shm (io_uring SQPOLL)	7.590	7.902	10.787	1518.780	15.439 ± 0.853
Message Size: 64 KiB — lower is better					
shm (busy_poll)	**11.848**	11.957	12.895	14.390	11.824 ± 0.008
shm (adaptive)	11.947	12.013	13.072	14.076	11.928 ± 0.008
shm (spin_back)	12.105	12.152	13.164	15.263	12.080 ± 0.008
shm (futex)	13.651	14.338	16.103	20.667	13.581 ± 0.017
shm (eventfd)	14.791	21.758	33.180	110.620	16.692 ± 0.381
unix_socket	14.793	17.163	19.825	24.540	15.713 ± 0.376
shm (io_uring)	14.854	14.999	16.704	21.883	15.023 ± 0.300
pipe	18.060	18.415	20.713	24.458	18.137 ± 0.016
posix_mq	N/A	N/A	N/A	N/A	N/A
shm (io_uring SQPOLL)	14.262	15.150	818.708	1623.410	28.588 ± 2.553
Message Size: 1 MiB — lower is better					
unix_socket	**122.353**	125.936	129.282	134.103	123.043 ± 0.086
shm (busy_poll)	134.712	139.938	146.149	152.020	135.735 ± 0.139
shm (spin_back)	138.311	149.811	160.231	169.930	140.585 ± 0.276
shm (adaptive)	140.755	151.248	203.329	1045.710	147.414 ± 2.376
shm (io_uring)	150.690	193.104	244.364	373.538	162.969 ± 1.159
shm (eventfd)	163.289	182.655	201.130	232.149	164.111 ± 0.738
shm (futex)	164.144	206.372	428.928	1124.960	175.985 ± 3.055
shm (io_uring SQPOLL)	145.616	819.006	1773.440	1938.180	282.603 ± 16.470
pipe	331.466	364.151	402.091	692.353	328.037 ± 1.937
posix_mq	N/A	N/A	N/A	N/A	N/A

(3) POSIX MQ is valid through 4 KiB in the recorded run. The repository's 4 KiB summary records 50,000 round trips (P50 4.408 µs). The 16 KiB and larger rows are explicitly marked unsupported by the host POSIX-MQ message-size limit and are reported as N/A.

(4) UNIX Socket wins at 1 MiB. 122.35 µs vs. 134.7–176.0 µs for SHM variants. On this host, the socket's buffering path outperforms the fixed-slot SHM configuration at this payload size, but the experiment does not isolate a kernel cause or repeat the comparison beyond 2 000 rounds. It is therefore a host-specific observation, not a general transport recommendation.

(5) SQPOLL-assisted signaling has low central latency but large tails. The SQPOLL-assisted signal-mode run completed all size-scaled Suite 2 samples, including 10 000 rounds at 64 KiB and 2 000 at 1 MiB. Its 64 B P50 is 6.912 µs, but its P99.9 reaches 1.485 ms. Since the blocking read/wait ring remains interrupt-mode to avoid deadlock, this result does not establish a fully syscall-free SQPOLL end-to-end path.

TABLE VI
STREAMING THROUGHPUT (GIB/S) — SATURATED REGIME, $N = 15$ RUNS

Wakeup Variant	64 B	4 KiB	64 KiB	1 MiB
busy_poll	0.316	10.17	**15.75**	**8.65**
spin_backoff	**0.344**	10.16	14.54	8.61
adaptive	0.338	10.43	15.53	8.36
eventfd	0.262	9.54	13.74	8.37
futex	0.300	**10.68**	15.60	8.18
io_uring	0.332	9.92	13.37	8.12
uring (SQPOLL)	0.109	4.22	6.66	4.45

B. Ablation: Wakeup Variant Comparison (Suite 2)

At 1 MiB, the six primary SHM variants span 134.7–176.0 μs (Table V). Copying 1 MiB across the ring dominates all wakeup mechanism overhead in this configuration. Wakeup selection may matter less when message service time is payload-dominated.

C. Streaming Throughput: Saturated Regime (Suite 1)

Table VI presents mean throughput; Figure 2 visualises the full size sweep.

Key observations:

(1) Saturated throughput is primarily data-path bound. Under fully saturated load the ring rarely empties, so the wakeup primitive is rarely invoked. The six variants remain in the same order of magnitude, with the controlled 64 KiB results spanning 13.37–15.75 GiB/s.

(2) The controlled io_uring result is 13.37 GiB/s at 64 KiB. This controlled result is used for the paper's throughput conclusions.

(3) Large messages converge. At 1 MiB the six controlled variants span 8.12–8.65 GiB/s. Payload copying and the fixed 64-slot ring dominate the wakeup mechanism at this size.

(4) Small-message io_uring overhead at 64 B. At 64 B payload, the controlled variants span 0.262–0.344 GiB/s. This variation reflects fixed per-message coordination overhead. At 4 KiB and above, payload copying dominates the observed throughput range.

(5) SQPOLL is reported separately. The single-channel SQPOLL configuration is slower throughout this saturated sweep (0.109–6.66 GiB/s over the reported sizes). This configuration-level result does not establish a general SQPOLL throughput limit; batching and poll-thread affinity remain outside the experiment.

D. Bursty Regime: Pure Wakeup Latency Ablation (Suite 1)

In the bursty regime, the consumer empties the ring and enters a sleep state before each burst. Table VII isolates wakeup latency per variant.

Frequency-controlled interpretation. The final bursty and offered-load runs were collected with the `performance` governor and Turbo disabled. This removes dynamic P-state frequency scaling as an explanation for the approximately

TABLE VII
BURSTY-REGIME RESULTS AT 64 B. ENTRIES ARE MEANS OF THE 15 RUN-LEVEL P50 OR CPU-TIME VALUES.

Variant	E2E P50	Wakeup P50	CPU μs/msg
busy_poll	0.260 μs	**0.022** μs	17.254
spin_back.	0.246 μs	0.094 μs	17.236
adaptive	126.008 μs	1029.26 μs	1.431
futex	126.357 μs	1091.92 μs	0.546
eventfd	126.277 μs	1093.81 μs	0.570
io_uring	126.658 μs	1094.98 μs	0.680
uring (SQPOLL)	5.130 μs	1058.47 μs	0.218

TABLE VIII
OFFERED-LOAD SWEEP AT 64 B: MEAN OF 15 RUN-LEVEL WAKEUP P50 VALUES (μS).

Variant	25%	50%	75%	90%	Sat.
busy_poll	**0.02**	**0.02**	**0.02**	**0.02**	**0.02**
spin_backoff	0.09	0.09	0.09	0.09	0.09
adaptive	1374.2	619.0	195.3	46.7	0.09
futex	1433.7	669.9	254.3	103.2	0.26
eventfd	1433.0	670.1	253.1	103.3	0.28
io_uring	1441.0	678.3	254.7	103.7	0.93
uring (SQPOLL)	1258.4	658.3	257.6	106.9	3.97

millisecond-scale low-load values. C-state residency and scheduler delay were not traced, so they remain plausible but unverified contributors to the measured end-to-end wakeup path. The close values for futex, eventfd, and interrupt-mode `io_uring` support the limited conclusion that no configuration has a decisive advantage for a single SPSC sleeping wakeup on this platform.

SQPOLL check. SQPOLL was run as a separately labelled configuration. At 64 B its bursty wakeup P50 was 1058.5 μs, versus 1095.0 μs for interrupt-mode `io_uring`; at 90% offered load the values were 106.9 and 103.7 μs, respectively. SQPOLL does not provide a material wakeup-latency improvement in this single-channel workload. Its complete 64 B comparison is included in Tables VII and VIII. The apparently discordant 5.130 μs SQPOLL E2E P50 in Table VII is not a 25$\times$ wakeup result: wakeup P50 times the empty-ring wait beginning before the next burst and therefore includes the intentional 1 ms inter-burst gap, whereas E2E P50 starts at each message's `slot.send_ns` publication timestamp and is calculated over all messages. The low SQPOLL E2E median consequently shows that most SQPOLL messages incurred little within-burst queue residence; because this suite does not trace SQ-thread service order, per-burst scheduler alignment, or queue residence for the other variants, the source of the unusually low E2E median remains unresolved. We therefore do not attribute it to an intrinsic SQPOLL wakeup advantage or use it as a comparative latency result.

As shown in Table VIII, under offered-load sweeps (25%, 50%, 75%, 90% of saturated capacity), spinning mechanisms (`busy_poll` and `spin_backoff`) maintain fixed

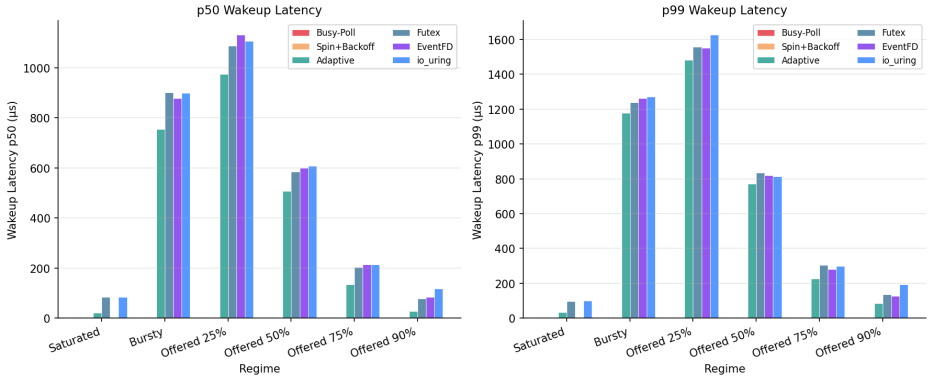

Fig. 1. Bursty-Regime Wakeup Latency Across Six SHM Variants. Spinning variants maintain near-zero wakeup latency while continuously spinning. Kernel-sleeping variants (`futex`, `eventfd`, `io_uring`) remain close at 1.092–1.095 ms in the frequency-pinned replication.

sub-microsecond latencies regardless of offered load. For kernel-sleeping variants (`futex`, `eventfd`, `io_uring`), median latency dynamically scales down as load increases—dropping from ≈1374–1441 μs at 25% load down to ≈103 μs at 90% load. The shorter inter-arrival gaps are consistent with less idle residency and scheduler descheduling, but this mechanism was not independently measured. At saturation, `io_uring`'s higher latency (0.93 μs) compared to `futex`/`eventfd` (0.26–0.28 μs) reflects the constant overhead of SQE/CQE ring management on the non-blocking fast path. SQPOLL follows the same load-dependent trend (1258.4 to 106.9 μs from 25% to 90%) but records 3.97 μs at saturation, so it is not a zero-overhead replacement in this configuration. `adaptive` hybrid spinning captures the best of both worlds, reducing low-load sleep penalties while achieving near-spinning performance (0.09 μs) under saturation; this describes latency and CPU time, not measured energy use.

The bursty-regime protocol (Table VII: 64-message burst, then a fixed 1 ms idle gap) and the offered-load sweep (Table VIII: periodic gaps calibrated to a fixed fraction of saturated rate) are deliberately different idle-gap structures and are not directly comparable message-for-message. The fixed 1 ms gap in the bursty protocol is shorter than the mean inter-arrival gap implied by 25% of saturated throughput at 64 B, so the CPU has less time to become idle before the next burst arrives—a plausible explanation for the lower ≈1090–1104 μs wakeup latency in Table VII relative to Table VIII's 25%-load row. As offered load rises toward saturation in Table VIII, the mean inter-arrival gap shrinks below the bursty protocol's fixed 1 ms gap, which is consistent with the 90%-load row (≈103 μs) falling below the bursty-regime figures despite both nominally representing "low load" conditions in isolation. The two tables should be read as separate experimental protocols with different idle-gap structure, not as demonstrations of a measured C-state ramp-up effect or repeated measurements of the same condition.

E. Controlled Throughput Dataset

The saturated-throughput results use a bit-identical 64-slot SPSC ring, size-scaled 16/128/512 MiB runs, and wakeup telemetry. Figure 2 and the throughput conclusions report this

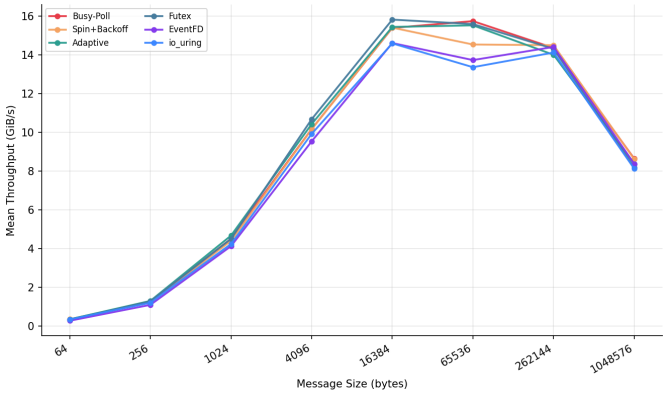

Fig. 2. Controlled saturated-throughput sweep for the six bit-identical SHM-ring wakeup variants ($N = 15$). This figure visualises the same data family as Table VI; SQPOLL is reported separately in that table.

TABLE IX
HARDWARE CACHE-MISS RATES FROM THE RECORDED PERF RUNS

Mechanism	L1 Miss %	LLC Miss %	Relative to Pipe
Pipe	2.42 %	33.6 %	1.00×
Socket	1.12 %	31.5 %	0.94×
MQ	N/A	3.2 %	0.10×
SHM Ring	3.97 %	4.97 %	0.15×

controlled ablation. Kernel-mediated transports are compared against the ring using the controlled depth-1 latency suite in Table V. An earlier direct-transport harness recorded a higher 64 KiB `io_uring` throughput value (28.17 GiB/s), but it used a different workload and lacks matching fixed-frequency environment records. Because the difference cannot be attributed to a single factor, that result is excluded from the paper's cross-mechanism conclusions.

F. Cache and Memory Hierarchy Analysis

The 4.97 % LLC miss rate is consistent with the SHM ring retaining a cache-friendly active working set. Pipe and socket record 30–34 % LLC miss rates in the corresponding perf runs. The `alignas(64)` layout also prevents `head` and `tail` from sharing a cache line.

VI. Discussion

A. What io_uring Actually Contributes to IPC

For a single SPSC ring channel, the completed depth-1 experiment shows that interrupt-mode `io_uring` does not reduce the measured per-event cost relative to the simpler blocking mechanisms. At 64 B it produces a 4.17 μs P50, versus `futex`'s 2.56 μs, consistent with SQE/CQE submission overhead. The bursty observations place all three kernel-sleeping variants in a similar range under recorded fixed-frequency settings. The SQPOLL result is also close, so this workload exposes no decisive wakeup-metric advantage over futex or eventfd; its anomalous E2E median remains unresolved and is not used for this comparison.

The primary performance advantage of SHM-based IPC originates entirely from replacing kernel-copy data paths with shared-memory `memcpy`—not from the wakeup primitive. Performance breaks down into a two-layer stack:

- **Data path** (dominant): shared memory vs. kernel-copy. It determines the observed throughput ordering at message sizes where payload transfer dominates.
- **Wakeup layer** (secondary): spinning vs. blocking. Dominates only when the ring is genuinely empty between bursts.

`io_uring`'s genuine advantages over `futex`/`eventfd` for this use case are: (1) a unified multi-FD event loop replacing $O(N)$ `futex_wait` calls with a single `io_uring_submit_and_wait`; (2) batched submission amortizing kernel-entry cost across multiple wakeup signals; and (3) a clear SQPOLL upgrade path that can remove the submission syscall via `IORING_SETUP_SQPOLL`.

B. Architectural Guidance by Use Case

a) High-Frequency Trading / Sub-μs Latency.: Use SHM ring with `busy_poll` or `adaptive`. Their 64 B P50 values are 185 and 211 ns, respectively, with P99 below 0.27 μs. Busy polling requires a dedicated spinning core; adaptive waiting blocks after its spin window. `adaptive` is preferred for intermittent workloads: it spins for up to 500 iterations and falls back to `futex` only when the ring is genuinely empty, reducing measured CPU time during burst-idle operation while preserving the hot path.

b) Cloud Microservices / CPU-Time-Constrained Environments.: Use SHM ring with `futex` or `eventfd`. Both use blocking waits and deliver approximately 2.6 μs P50 and sub-4.2 μs P99 at 64 B in depth-1 ping-pong. `eventfd` integrates naturally into existing `poll`/`epoll` event loops without futex-specific code, making it the lower-friction choice for retrofit into existing stacks.

c) Async Event-Driven Frameworks.: Use SHM ring with `io_uring`. Wakeup latency matches `futex`/`eventfd` within the same kernel-sleeping class (2.6–4.2 μs P50), but the `io_uring` context unifies IPC ring monitoring with network I/O, disk I/O, and timer events in a single `submit_and_wait` call. This eliminates the complexity of maintaining parallel `epoll` and `futex` contexts and provides a clear upgrade path to SQPOLL mode.

d) Large Payloads (>1 MiB) or Deep Buffering.: Treat the UNIX-socket result as a follow-up candidate, not a default choice. At 1 MiB, UNIX Socket achieves 122.4 μs P50 versus 134.7–176.0 μs for SHM ring variants on this host. Because the comparison has only 2 000 rounds and lacks copy-path and syscall profiling, its cause and generality remain unestablished.

C. Ring-Depth Constraint at 1 MiB

At 1 MiB, the controlled saturated ablation variants span 8.12–8.65 GiB/s. This convergence is consistent with the fixed 64-slot ring geometry, although the current experiment does not include a controlled ring-depth sweep. With each slot sized at 1 MiB, the ring holds at most 64 MiB of in-flight data. The producer stalls as soon as all 64 slots are occupied while the consumer processes each 1 MiB slot sequentially.

The fixed $N = 64$ slot count therefore bounds in-flight buffering ($64 \times 1\,\text{MiB} = 64\,\text{MiB}$). The present measurements do not separate that capacity effect from copy bandwidth, checksum cost, or socket buffering, so the ring-depth explanation remains a hypothesis rather than a demonstrated causal result.

No controlled interrupt-mode outlier. In the controlled saturated ablation, interrupt-mode `io_uring` reaches 8.12 GiB/s at 1 MiB, within the 8.12–8.65 GiB/s range of all six variants.

Increasing `NUM_SLOTS` to 256 would raise in-flight capacity to 256 MiB. A controlled depth sweep is required to determine whether that change recovers throughput at 1 MiB; this is left as future work.

D. Comparison with Related Measurement Studies

Didona et al. [7] compare `libaio`, SPDK, and `io_uring` for storage I/O, showing that the relative cost of an I/O API depends on workload and execution model. Their study does not evaluate IPC; independently, our depth-1 IPC ping-pong benchmark (Suite 2) finds that per-message submission overhead makes `io_uring` slightly slower than `futex`, with wakeups amortized only across bursty arrivals (Suite 1). In contrast to Ren and Trivedi [8], who attribute speedups to `io_uring`'s reduced per-operation overhead for NVMe I/O, our experiment shows this benefit does not transfer to shared-memory IPC wakeup: the NVMe path benefit arises from batching DMA descriptor submissions, whereas SPSC wakeup is inherently one-at-a-time (one signal per empty-ring event). Batching is only applicable when multiple channels empty simultaneously, which requires a multi-ring architecture outside the scope of this study.

VII. Threats to Validity

1) **Single node and microarchitecture.** All experiments ran on one Intel 12th-Gen x86_64 laptop. Results on ARM64, AMD Zen, or multi-socket NUMA may differ due to different cache-coherency protocols and scheduler implementations. The host has hybrid P/E cores; logical CPU IDs were pinned, but their core class was not independently recorded, limiting interpretation of scheduler-sensitive tail latency.

2) **SPSC topology only.** Results apply to Single-Producer Single-Consumer queues. MPMC rings require CAS loops introducing contention absent in SPSC.
3) **Synthetic payload workload.** Payloads consist of repeated-byte patterns. Real application payloads (protobuf, encrypted frames) may shift the bottleneck to serialisation rather than ring mechanics.
4) **Residual idle-state and scheduler effects.** The final runs record `performance` and `no_turbo=1`, but do not use `isolcpus`/`nohz_full`. Thus, the result is specific to this native Linux laptop and should not be generalized to all CPUs.
5) **Suite 1 clock placement.** Suite 1 latency fields use cross-core `high_resolution_clock` timestamps. The corrected depth-1 Suite 2 avoids this limitation by placing both `CLOCK_MONOTONIC_RAW` timestamps on the initiator core.
6) **POSIX MQ coverage.** The recorded configuration includes a valid 4 KiB run. The 16 KiB and larger rows are marked `unsupported` because of the host message-size limit.
7) **SQPOLL scope.** SQPOLL is evaluated as a separate 64-slot, single-channel configuration. Its result does not generalize to multi-FD batching or other SQPOLL thread-affinity choices. The completed Suite 2 SQPOLL-assisted signal-mode run uses the reference host, but retains interrupt-mode blocking reads to avoid deadlock. Its anomalously low Suite 1 E2E median is not used as a comparative result because the current instrumentation cannot explain its queue-residence difference.
8) **Tail-percentile provenance.** Table V is derived only from the root `data/pingpong_*_summary.csv` files. POSIX MQ has no valid 1 MiB result. The 1 MiB distributions contain 2 000 rounds, so their P99.9 estimates are exploratory (approximately two samples lie beyond that percentile).

VIII. Conclusion & Future Work

This paper presented a rigorous, controlled measurement study of wakeup mechanism performance in lock-free shared-memory SPSC IPC. By holding the ring buffer design constant and varying only the sleep/wakeup primitive, we isolated the true contribution of `io_uring` to IPC latency.

Summary of findings:

1) **The shared-memory data path drives the performance gains.** Spinning on a cache-aligned ring delivers 185 ns P50 at 64 B. In the controlled saturated ablation, the six primary variants reach 13.37–15.75 GiB/s at 64 KiB.
2) **Interrupt-mode wakeup result is deliberately scoped.** In strict depth-1 ping-pong, interrupt-mode `io_uring` pays a modest per-event overhead relative to futex. The frequency-pinned bursty and offered-load replication shows closely grouped sleeping variants, while SQPOLL also does not materially improve the 64 B wakeup result in this single-channel design.
3) **Spinning variants dominate sub-μs latency requirements.** `adaptive` balances latency and CPU time: it achieves 211 ns P50 and lowers recorded CPU time during idle periods relative to continuous busy polling. This study does not measure energy or package power.
4) **The 1 MiB throughput convergence may reflect ring depth.** A controlled ring-depth sweep is needed to test whether increasing `NUM_SLOTS` recovers the throughput advantage.
5) **Cache hierarchy alignment is critical.** The SHM ring achieves a 4.97 % LLC miss rate vs. 30–34 % for kernel-copy mechanisms, consistent with reduced cache pressure in the shared-memory data path.

Future directions:

- *Multi-FD SQPOLL mode.* Evaluate whether batching several channels or changing the SQPOLL thread affinity changes the single-channel result reported here.
- *MPMC extension.* A lock-free MPMC ring with sequence-lock or CAS-based reservation extends the ablation to multi-producer settings.
- *Cross-NUMA evaluation.* Profiling with producer on NUMA node 0 and consumer on NUMA node 1 would quantify LLC miss penalties and motivate NUMA-aware ring partitioning.
- *RDMA baseline.* An InfiniBand one-sided write baseline contextualises these results within HPC cluster IPC.
- *Production traffic models.* Replacing synthetic payloads with real-world Kafka message logs or gRPC frame traces would validate whether serialisation overhead shifts the bottleneck off ring mechanics.

References

[1] J. Axboe, "Efficient I/O with io_uring," Linux Kernel Documentation, Tech. Rep., 2019. Available: https://kernel.dk/io_uring.pdf
[2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley Professional, 2013.
[3] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
[4] L. Lamport, "Specifying concurrent program modules," *ACM Trans. Programming Languages and Systems*, vol. 5, no. 2, pp. 190–222, Apr. 1983.
[5] M. Thompson, D. Farley, M. Barker, P. Gee, and A. Stewart, "Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads," LMAX Exchange, White Paper, 2011.
[6] DPDK Project, "DPDK Programmer's Guide: Ring Library," 2024. Available: https://doc.dpdk.org/guides/prog_guide/ring_lib.html
[7] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, "Understanding modern storage APIs: A systematic study of libaio, SPDK, and io_uring," in *Proc. 15th ACM International Systems and Storage Conference (SYSTOR '22)*, Haifa, Israel, Jun. 2022, pp. 1–16.
[8] Z. Ren and A. Trivedi, "Performance Characterization of Modern Storage Stacks: POSIX I/O, libaio, SPDK, and io_uring," in *Proc. 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems (CHEOPS '23)*, Rome, Italy, May 2023.
[9] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.
[10] Linux man-pages project, `futex(2)`, `eventfd(2)`, `io_uring(7)`, `mq_overview(7)`, `pipe(7)`, `unix(7)`, kernel.org, 2024. Available: https://man7.org/linux/man-pages/

Reproducibility. All source code, benchmark scripts, raw CSV data, and figure-generation scripts are publicly available at https://github.com/Rahul09123/io_uring-IPC.